

Functions, methods, structs, and interfaces

Deep reference for Go's behavior side: how code is organized into callable units and composable types.

Functions

Signature forms

```
func add(a, b int) int { return a + b }
func swap(a, b string) (string, string) { return b, a }
func split(n int) (x, y int) {                // named returns
    x = n / 2
    y = n - x
    return                                     // naked return
}
func log(prefix string, args ...any) { }      // variadic
func noop() { }                               // no params, no return
```

Multiple returns

Idiomatic Go. The last return is conventionally an error.

```
func divide(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("divide by zero")
    }
    return a / b, nil
}

q, err := divide(10, 2)
if err != nil { return err }
```

Named returns

The return variables are pre-declared as local variables. A bare `return` returns their current values.

```
func parse(s string) (n int, err error) {
    n, err = strconv.Atoi(s)
    return
}
```

Useful when: - You want `defer` to mutate the return (e.g., wrap an error). - Documentation benefits from naming the values.

Skip them when: - The function is small and obvious. - Naked returns become unclear (more than 2-3 lines into the body).

Variadic

```
func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}

sum(1, 2, 3)
sum() // empty: nums is nil
nums := []int{1, 2, 3}
sum(nums...) // spread (the slice IS the parameter, not a copy)
```

`nums` inside the function is a `[]int`. When you spread, no new slice is allocated; mutations inside the function affect the caller's slice.

Functions are first-class values

```
var f func(int) int = double
f(10) // 20

// pass as parameter
func apply(g func(int) int, x int) int { return g(x) }

// return a function
func adder(x int) func(int) int {
    return func(y int) int { return x + y }
}

add5 := adder(5)
add5(3) // 8
```

Function types are based on signature. `func(int) int` and `func(int) int` are the same type regardless of which function fills the slot.

Closures

A function literal captures variables from the enclosing scope by reference.

```
func counter() func() int {
    n := 0
    return func() int { n++; return n }
}

c1 := counter()
c1(); c1() // 1, 2
c2 := counter()
c2() // 1 (separate state)
```

The captured `n` lives on the heap because the closure outlives the function call. Escape analysis decides this automatically.

Closures are how Go does callbacks, option functions, deferred actions, and per-request state.

Recursion

Plain recursion works. No tail-call optimization, so deep recursion can blow the stack. Go stacks grow dynamically (start at 2-8 KB, expand as needed), so the practical limit is high, but iterative is still safer for unbounded depth.

```
func factorial(n int) int {
    if n <= 1 { return 1 }
    return n * factorial(n-1)
}
```

defer

`defer` schedules a call to run when the enclosing function returns.

```
func read(path string) error {
    f, err := os.Open(path)
    if err != nil { return err }
    defer f.Close()
    // ...
    return nil
}
```

Key rules: 1. LIFO order across multiple defers. 2. Arguments are evaluated when `defer` is reached, not when the call runs. 3. Deferred closures can mutate named return values. 4. Don't `defer` inside a long loop; resources accumulate.

```
func wrap() (err error) {
    defer func() {
        if err != nil {
            err = fmt.Errorf("wrap: %w", err)
        }
    }()
    return doStuff()
}
```

panic and recover

`panic` halts execution and unwinds the stack, running deferred calls along the way. `recover` (inside a deferred function) stops the unwind.

```
func safe() (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("panic: %v", r)
        }
    }()
    risky()
    return nil
}
```

Reserve panic for true bugs (nil deps at startup, “should never happen” invariants). Don’t use it for normal errors. Don’t recover unless you have a specific reason (e.g., a server boundary that shouldn’t let one bad request crash the process).

Methods

A method is a function with a receiver. The receiver binds the function to a type.

```
type Circle struct { R float64 }

func (c Circle) Area() float64 {
    return math.Pi * c.R * c.R
}

c := Circle{R: 5}
c.Area()
```

Value vs pointer receiver

```
func (c Circle) Area() float64 { ... }    // value receiver
func (c *Circle) Scale(f float64) {      // pointer receiver
    c.R *= f
}
```

Differences:

	Value receiver	Pointer receiver
Copies the receiver	Yes	No
Can mutate the receiver	No	Yes
Works on addressable values	Yes	Yes
Works on map values	Yes	No (map elements aren't addressable)
Method set includes both?	Value: only value methods. Pointer: both.	Same

Choosing the receiver type

- Mutates the receiver? Pointer.
- Large struct (more than a few words)? Pointer (cheaper to call).
- Contains a `sync.Mutex` or similar? Pointer (copying defeats the lock).
- Consistency: if any method needs pointer receiver, make all of them pointer for that type.

When in doubt, pointer receiver.

Method sets

The method set of a type determines which interfaces it satisfies.

- Type `T` has methods with receiver `T`.
- Type `*T` has methods with receiver `T` and `*T`.

```
type Animal interface { Speak() }

type Dog struct{}
func (d *Dog) Speak() { fmt.Println("woof") }

var a Animal = &Dog{}    // works: *Dog has Speak
var a Animal = Dog{}     // compile error: Dog has no Speak (it's on *Dog)
```

Methods on named types

You can define methods on any named type you own, not just structs.

```
type Celsius float64

func (c Celsius) Fahrenheit() Celsius {
    return c*9/5 + 32
}

c := Celsius(100)
c.Fahrenheit()           // 212
```

You can't add methods to types from other packages. Wrap them in your own named type if you need to.

Methods cannot be added to other packages' types

```
// in your package
func (s string) Reverse() string { ... }    // compile error: cannot define methods
                                           on non-local type
```

The workaround:

```
type MyString string
func (s MyString) Reverse() string { ... }
```

Method expressions and method values

```
c := Circle{R: 5}

// method value: receiver is captured
f := c.Area
f()                      // calls c.Area()

// method expression: receiver becomes the first argument
g := Circle.Area
g(c)                     // calls Circle.Area(c)
```

Useful for passing methods to higher-order functions like `sort.Slice` or as callbacks.

Pointers

A pointer holds the memory address of a value.

```

x := 10
p := &x           // pointer to x
*p = 20           // dereference and assign
fmt.Println(x)    // 20

var q *int         // zero value: nil
*q               // panic: nil dereference

```

Operators

Operator	Meaning
<code>&x</code>	address of <code>x</code>
<code>*p</code>	value at <code>p</code> (dereference)
<code>*T</code>	type “pointer to T”

Pointer to struct

```

p := &Point{X: 1, Y: 2}
p.X           // shorthand for (*p).X (Go auto-dereferences)
(*p).X        // also works, but no one writes this

```

Method calls auto-take-address too:

```

p := Point{X: 1}           // value, not pointer
p.Move(1, 1)               // if Move has *Point receiver, Go takes &p
                          // automatically
                          // works because p is addressable

```

Value semantics by default

Function parameters are pass-by-value. To mutate the caller’s variable, pass a pointer.

```

func inc(x int) { x++ }    // no effect on caller
func inc(x *int) { *x++ }  // mutates caller

n := 0
inc(&n)
// n == 1

```

Reference-like types

Slices, maps, channels, functions, and interfaces internally hold pointers. Passing one copies the small header but shares the underlying data.

```
func grow(m map[string]int) { m["a"] = 1 } // visible to caller
func grow(s []int) { s[0] = 99 } // visible to caller (shares backing array)
```

But reassigning the local variable doesn't affect the caller:

```
func reset(s []int) { s = nil } // no effect on caller's slice
```

new vs &T

```
p := new(Point) // *Point, zero-valued
p := &Point{} // same, more flexible (you can set fields)
p := &Point{X: 1, Y: 2} // with values
```

`new(T)` allocates and returns `*T`. It exists mostly for legacy reasons. Prefer `&T{}` because you can populate fields in the same expression.

Nil pointers

```
var p *User // nil
p.Name // panic: nil dereference

if p != nil {
    fmt.Println(p.Name)
}
```

But: methods can be defined on `*T` and called on a nil pointer, as long as the method doesn't dereference.

```
func (u *User) ID() string {
    if u == nil { return "" }
    return u.id
}
```

Escape analysis

The compiler decides whether a value lives on the stack or the heap. If a pointer to a local escapes the function (returned, captured by a closure, stored in a heap object), the value moves to the heap.

```
func newUser() *User {
    u := User{} // escapes: pointer returned
    return &u
}
```


You don't manage this. Heap allocations are garbage collected. Inspect with `go build -gcflags=-m`.

Address-of restrictions

Not everything is addressable. You can't take the address of:

```
&5                // literal
&("hello")        // string literal
&f()              // function return value
m := map[string]int{}
&m["k"]           // map element (workaround: pull into variable)
```

Variables, struct fields of addressable values, slice elements, and pointer dereferences are addressable.

Pointers vs references

Go has no reference type. Every pointer is explicit. C++ `&` references and Java auto-references don't exist here.

When to use a pointer

- The function needs to mutate the value.
- The value is large enough that copying matters (rough threshold: more than 64 bytes).
- The type holds a `sync.Mutex`, channel buffer, or other reference that must stay shared.
- The value semantically represents an entity with identity (a user, a request, a connection).
- The field is optional and you need to distinguish "absent" from "zero".

When to use a value: - Small, copyable, no identity (a Point, a Color, a Duration). - Read-only methods.
- Immutability is part of the contract.

Structs

A struct groups named fields into one type.

```
type User struct {
    ID    int
    Name  string
    Email string
}

u := User{ID: 1, Name: "alice", Email: "a@x.com"} // named fields
u := User{1, "alice", "a@x.com"}                 // positional (fragile)
u := User{}                                       // zero value
p := &User{Name: "alice"}                       // pointer literal
```

Field access

```
u.Name
u.Name = "bob"
p.Name           // auto-deref through pointer
```

Anonymous structs

For one-off composite types, often in tests, JSON shaping, or table-driven test cases.

```
data := struct {
    Status int
    Body   string
}{200, "ok"}

// table-driven test
cases := []struct {
    in, want string
}{
    {"a", "A"},
    {"b", "B"},
}
```

Embedded fields

A field declared with just a type (no name) is embedded. Its fields and methods are promoted to the outer struct.

```
type Animal struct { Name string }
func (a Animal) Greet() string { return "hi " + a.Name }

type Dog struct {
    Animal           // embedded
    Breed string
}

d := Dog{Animal: Animal{Name: "Rex"}, Breed: "Lab"}
d.Name           // promoted from Animal
d.Greet()        // promoted method

// you can still reach the inner explicitly
d.Animal.Name
```

Embedding is composition. Go has no inheritance. The outer type doesn't override the inner unless explicitly redefined.

If the outer type defines a method with the same name, it shadows the embedded one. Calls on the outer use the outer's; the embedded one is still reachable via the explicit path.

Multiple embedding

```
type A struct{}; func (A) Hello() {}
type B struct{}; func (B) Hello() {}

type C struct { A; B }

c := C{}
c.Hello()           // compile error: ambiguous
c.A.Hello()         // OK
```

Struct tags

Metadata read at runtime by encoders, validators, and ORMs.

```
type User struct {
    ID    int    `json:"id"`
    Email string `json:"email,omitempty" validate:"required,email"`
    PW    string `json:"- "`           // skip in JSON
}
```

The tag is a string. Convention: space-separated `key:"value"` pairs. Read via reflection in encoder libraries.

Comparability

A struct is comparable if all its fields are comparable. Then `==` does field-by-field comparison.

```
type Point struct { X, Y int }
Point{1, 2} == Point{1, 2}      // true

type Bad struct { S []int }
Bad{} == Bad{}                  // compile error: slice in struct
```

Structs of comparable types are valid map keys.

Empty struct

`struct{}` takes zero bytes. Used for set-like maps and for signal-only channels.

```
set := map[string]struct{}{}
set["a"] = struct{}{}
_, has := set["a"]

done := make(chan struct{})
close(done)           // signal, no data
```

Memory layout and padding

Fields are stored in declared order. The compiler inserts padding so each field is aligned to its size. Reordering fields by size (largest first) can shrink the struct.

```
type Bad struct {
    a bool    // 1 byte + 7 bytes padding (align int64)
    b int64   // 8 bytes
    c bool    // 1 byte + 7 bytes padding (align next struct)
}
// total: 24 bytes

type Good struct {
    b int64   // 8
    a bool    // 1
    c bool    // 1 + 6 padding
}
// total: 16 bytes
```

`unsafe.Sizeof(x)` reports the size. Only worth optimizing for high-count structs in hot paths.

Interfaces

An interface is a set of method signatures. Any type that implements all the methods satisfies the interface, automatically. No `implements` keyword.

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type File struct { ... }
func (f *File) Read(p []byte) (int, error) { ... }

var r Reader = &File{}           // satisfies Reader implicitly
```

Implicit satisfaction

Decoupling: the implementer doesn't depend on the interface package. Define interfaces where they're used (consumer side), not where they're implemented. This is the Go inversion of dependency injection: the caller declares what it needs.

Method sets and pointer receivers

Receiver	Methods in T's set	Methods in *T's set
<code>func (t T) M()</code>	M	M
<code>func (t *T) M()</code>	(none)	M

If a method has pointer receiver, only `*T` satisfies an interface requiring it.

```
type Speaker interface { Speak() }
type Dog struct{}
func (d *Dog) Speak() {}

var s Speaker = Dog{}           // compile error
var s Speaker = &Dog{}         // OK
```

Empty interface and any

`interface{}` (alias `any` from Go 1.18) is satisfied by every type.

```
func print(v any) { fmt.Println(v) }
```

Use for true heterogeneous containers (JSON decoding into unknown shapes, formatting). For function parameters where types are knowable, use generics instead.

Type assertions

Pull a concrete value out of an interface.

```
var i any = "hello"
s := i.(string)           // panics if i isn't a string
s, ok := i.(string)       // safe form
```

Type switches

Multi-way dispatch by dynamic type.

```

switch v := i.(type) {
case nil:
    fmt.Println("nil")
case int:
    fmt.Println("int", v)
case string:
    fmt.Println("string", v)
case fmt.Stringer:
    fmt.Println("stringer", v.String())
default:
    fmt.Printf("unknown %T", v)
}

```

Interface composition

Build larger interfaces from smaller ones.

```

type Reader interface { Read([]byte) (int, error) }
type Closer interface { Close() error }

type ReadCloser interface {
    Reader
    Closer
}

```

`io.ReadCloser`, `io.ReadWriter`, etc. are built this way.

nil interface vs interface holding a nil pointer

```

var p *MyError = nil
var err error = p
err == nil // FALSE

```

An interface is `nil` only if both the type and value parts are nil. Assigning a typed nil pointer gives you an interface with a type but no value. The `!= nil` check then returns true. This is one of the most-cited Go gotchas.

Fix: return concrete `nil` from functions that can return errors.

```

func doSomething() error {
    var err *MyError = nil
    if cond { err = &MyError{} }
    if err != nil { return err } // returns typed nil, breaks caller's nil check
    return nil // good
}

```

Compile-time interface checks

To assert that a type satisfies an interface at compile time:

```
var _ io.Reader = (*MyReader)(nil)
```

The blank assignment exists only for type-checking. Catches missing method bugs at build time, not runtime.

Common standard library interfaces

Interface	Method(s)
error	Error() string
fmt.Stringer	String() string
io.Reader	Read([]byte) (int, error)
io.Writer	Write([]byte) (int, error)
io.Closer	Close() error
io.ReaderFrom	ReadFrom(r Reader) (n int64, err error)
io.WriterTo	WriteTo(w Writer) (n int64, err error)
sort.Interface	Len, Less, Swap
json.Marshaler	MarshalJSON() ([]byte, error)
json.Unmarshaler	UnmarshalJSON([]byte) error
context.Context	Deadline, Done, Err, Value

Implementing common interfaces makes your type compose with the standard library.

Interface design rules

1. **Small interfaces.** 1-3 methods. `io.Reader` is the gold standard.
2. **Define on the consumer side.** The function that takes the interface declares it, not the type that implements it.
3. **Accept interfaces, return concrete types.** Callers can pass anything; readers know exactly what they get back.
4. **Don't add a method to an interface for one caller.** Make a new interface.
5. **Don't define an interface "just in case."** Define when you have at least two implementations or a real testing need.

Best practices

1. **Pointer receiver if mutating, large struct, or contains a mutex.** Otherwise value.
 2. **Consistency: pick one receiver type per type.** Don't mix value and pointer receivers on the same struct.
 3. **Wrap errors with context.** `fmt.Errorf("opening %s: %w", path, err)`.
 4. **Use named return values when `defer` mutates the return** (typically error wrapping).
 5. **Embed for composition, not inheritance.** If you find yourself fighting the embed semantics, switch to an explicit field.
 6. **Interfaces small and consumer-side.** A 7-method interface is usually two interfaces.
 7. **Avoid `init()`** . Hides side effects, complicates testing.
 8. **Pass `context.Context` first.** Methods that block or do I/O take `ctx context.Context` as the first param.
 9. **Never return typed nil as `error`** . Return literal `nil` from the function.
 10. **Use `any` only when you mean it.** If types are known, use generics or a small interface.
-

Common gotchas

Gotcha	Symptom	Fix
Value receiver mutates struct	Caller sees no change	Pointer receiver
Mixed value/pointer receivers	Surprising method-set behavior	Pick one and stick to it
Interface holds typed nil	<code>err != nil</code> returns true unexpectedly	Return concrete <code>nil</code>
Map value method call mutates	No effect	Pull into a variable, mutate, write back
Embedded method shadowed silently	Outer method called instead	Read carefully; use explicit path
Spread of nil slice into variadic	Works fine (length 0)	Not actually a bug
Closure captures loop variable (Go <1.22)	All goroutines see same value	<code>x := x</code> rebind or pass as arg
Named return with shadowing inside	Wrong value returned	Use <code>=</code> to assign to the named return
<code>new(T)</code> then trying to set fields	Awkward syntax	<code>&T{Field: ...}</code>
Comparing structs with slice fields	Compile error	Implement <code>Equal</code> method
Defining method on imported type	Compile error	Wrap in a local named type
Address of map element	Compile error	Pull into variable; use <code>map[K]*V</code>
Returning interface, accepting struct	Hard to test, harder to swap	Flip it: accept interface, return struct
Big interface	Hard to mock	Split into smaller ones
<code>defer</code> in loop accumulates	Resources leak	Extract to helper function

Quick reference

What	Syntax
Function	<code>func f(a, b int) int</code>
Multiple return	<code>func f() (int, error)</code>
Named return	<code>func f() (n int, err error)</code>
Variadic	<code>func f(xs ...int)</code>
Function value	<code>var f func(int) int</code>
Closure	<code>func() int { return x }</code>
Defer	<code>defer f()</code>
Method (value)	<code>func (t T) M()</code>
Method (pointer)	<code>func (t *T) M()</code>
Method expression	<code>T.M</code>
Method value	<code>t.M</code>
Pointer to T	<code>*T</code>
Address of	<code>&x</code>
Dereference	<code>*p</code>
Allocate	<code>new(T) or &T{}</code>
Struct literal	<code>T{Field: value}</code>
Anonymous struct	<code>struct{X int}{X: 1}</code>
Embedded field	<code>type Dog struct { Animal; Breed string }</code>
Empty struct	<code>struct{}{}</code>
Struct tag	<code>`json:"name"`</code>
Interface	<code>type I interface { M() }</code>
Type assertion	<code>v, ok := i.(T)</code>
Type switch	<code>switch v := i.(type)</code>

What	Syntax
Compile-time check	<code>var _ I = (*T)(nil)</code>
Any	<code>any</code> (alias for <code>interface{}</code>)
Interface composition	<code>type RW interface { Reader; Writer }</code>